

Reshaping data in pandas

Exam information

- Exam 1 is next Friday (10/2)
- Covers data wrangling in R and Python (heavier emphasis on R)
 - Material up through 9/25 (this Friday)
- What I might ask:
 - Provide output for a given chunk of code
 - Write a small chunk of code to produce desired output
 - Explain data wrangling steps you would take to answer a research question
- Current HW has some questions that are some examples of exam-type questions

Last time

Key difference between R (tidyverse) and Python (pandas):

- Series and DataFrames in pandas always have an index!
- We can index Series and DataFrames by index label, or by their position

Today: some other wrangling tools in pandas, and thinking more about the index

Grouping and aggregating

Question: What is the index when we group and aggregate?

```
1 (penguins.groupby('species')
2     .agg(
3         mean_bill = ('bill_length_mm', 'mean'),
4         sd_bill = ('bill_length_mm', 'std')
5     ))
```

	mean_bill	sd_bill
species		
Adelie	38.791391	2.663405
Chinstrap	48.833824	3.339256
Gentoo	47.504878	3.081857

Grouping and aggregating

```
1 penguins_summary = (penguins.groupby('species')
2     .agg(
3         mean_bill = ('bill_length_mm', 'mean'),
4         sd_bill = ('bill_length_mm', 'std')
5     ))
6
7 penguins_summary
```

index
↓

species	mean_bill	sd_bill
Adelie	38.791391	2.663405
Chinstrap	48.833824	3.339256
Gentoo	47.504878	3.081857

What if I just want the Adelies?

Grouping and aggregating

```
1 penguins_summary = (penguins.groupby('species')
2     .agg(
3         mean_bill = ('bill_length_mm', 'mean'),
4         sd_bill = ('bill_length_mm', 'std')
5     ))
6
7 penguins_summary
```

	mean_bill	sd_bill
species		
Adelie	38.791391	2.663405
Chinstrap	48.833824	3.339256
Gentoo	47.504878	3.081857

```
1 penguins_summary.loc['Adelie']
```

```
mean_bill    38.791391
sd_bill       2.663405
Name: Adelie, dtype: float64
```

Grouping and aggregating

What is the index now?

```
1 (penguins.groupby(['species', 'island'])
2     .agg(
3         mean_bill = ('bill_length_mm', 'mean'),
4         sd_bill = ('bill_length_mm', 'std')
5     ))
```

		mean_bill	sd_bill
species	island		
Adelie	Biscoe	38.975000	2.480916
	Dream	38.501786	2.465359
	Torgersen	38.950980	3.025318
Chinstrap	Dream	48.833824	3.339256
Gentoo	Biscoe	47.504878	3.081857

Grouping and aggregating

```
1 (penguins.groupby(['species', 'island'])
2     .agg(
3         mean_bill = ('bill_length_mm', 'mean'),
4         sd_bill = ('bill_length_mm', 'std')
5     )).index
```

```
MultiIndex([( 'Adelie',    'Biscoe'),
            ( 'Adelie',    'Dream'),
            ( 'Adelie', 'Torgersen'),
            ('Chinstrap',    'Dream'),
            ( 'Gentoo',    'Biscoe')],
            names=['species', 'island'])
```

MultiIndex

```
1 penguin_summary = (penguins.groupby(['species', 'island'])
2     .agg(
3         mean_bill = ('bill_length_mm', 'mean'),
4         sd_bill = ('bill_length_mm', 'std')
5     ))
6
7 penguin_summary
```

		mean_bill	sd_bill
species	island		
Adelie	Biscoe	38.975000	2.480916
	Dream	38.501786	2.465359
	Torgersen	38.950980	3.025318
Chinstrap	Dream	48.833824	3.339256
Gentoo	Biscoe	47.504878	3.081857

```
1
2 penguin_summary.loc['Adelie']
```

	mean_bill	sd_bill
island		
Biscoe	38.975000	2.480916
Dream	38.501786	2.465359
Torgersen	38.950980	3.025318

MultiIndex

```
1 penguin_summary = (penguins.groupby(['species', 'island'])
2     .agg(
3         mean_bill = ('bill_length_mm', 'mean'),
4         sd_bill = ('bill_length_mm', 'std')
5     ))
6
7 penguin_summary
```

		mean_bill	sd_bill
species	island		
Adelie	Biscoe	38.975000	2.480916
	Dream	38.501786	2.465359
	Torgersen	38.950980	3.025318
Chinstrap	Dream	48.833824	3.339256
Gentoo	Biscoe	47.504878	3.081857

```
1
2 penguin_summary.loc[('Adelie', 'Biscoe')]
```

```
mean_bill    38.975000
sd_bill       2.480916
Name: (Adelie, Biscoe), dtype: float64
```

Avoiding changing the index

```
1 (penguins.groupby(['species', 'island'],  
2                   as_index = False) ← don't use grouping variables in Index  
3     .agg(  
4       mean_bill = ('bill_length_mm', 'mean'),  
5       sd_bill = ('bill_length_mm', 'std')  
6     ))
```

	species	island	mean_bill	sd_bill
0	Adelie	Biscoe	38.975000	2.480916
1	Adelie	Dream	38.501786	2.465359
2	Adelie	Torgersen	38.950980	3.025318
3	Chinstrap	Dream	48.833824	3.339256
4	Gentoo	Biscoe	47.504878	3.081857

Reshaping data

```
# A tibble: 260 × 38
```

```
  country `1975` `1976` `1977` `1978` `1979` `1980` `1981` `1982`  
`1983` `1984`  
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  
<dbl> <dbl>  
1 Afghan...    NA    NA    NA    NA    4.99    NA    NA    NA  
NA    NA  
2 Albania     NA    NA    NA    NA    NA     NA    NA    NA  
NA    NA  
3 Algeria     NA    NA    NA    NA    NA     NA    NA    NA  
NA    NA  
4 Andorra     NA    NA    NA    NA    NA     NA    NA    NA  
NA    NA
```

Question: How do we want to reshape this data?

<u>Country</u>	<u>Year</u>	<u>Literacy</u>
Afghanistan	1975	NA
⋮	⋮	⋮

Tidy data

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174604898
China	1999	212258	1272915272
China	2000	216766	1280425583

variables

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174604898
China	1999	212258	1272915272
China	2000	216766	1280425583

observations

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174604898
China	1999	212258	1272915272
China	2000	216766	1280425583

values

(Image from *R for Data Science*)

In R: pivot_longer

```
1 litF |>
2   pivot_longer(
3     cols = -country,
4     names_to = "year",
5     values_to = "literacy_rate"
6   ) |>
7   drop_na()
```

A tibble: 571 × 3

	country <chr>	year <chr>	literacy_rate <dbl>
1	Afghanistan	1979	4.99
2	Afghanistan	2011	13
3	Albania	2001	98.3
4	Albania	2008	94.7
5	Albania	2011	95.7
6	Algeria	1987	35.8
7	Algeria	2002	60.1
8	Algeria	2006	63.9
9	Angola	2001	54.2
10	Angola	2011	58.6

In pandas: melt

columns not to pivot

line names to in R
values to in R

```
1 (litF.melt(id_vars = 'country',  
2           var_name = 'year',  
3           value_name = 'literacy_rate')  
4           .dropna(axis = 0))
```

	country	year	literacy_rate
30	Burkina Faso	1975	3.182766
37	Central African Rep.	1975	8.399576
99	Kuwait	1975	48.015214
191	Turkey	1975	45.098921
197	United Arab Emirates	1975	38.124870
...	...	...	...
9562	Vanuatu	2011	81.553540
9564	West Bank and Gaza	2011	92.616180
9565	Vietnam	2011	91.383460
9566	Yemen, Rep.	2011	48.539050
9568	Zimbabwe	2011	80.065659

In pandas: melt

```
1 (litF.melt(id_vars = 'country',  
2           var_name = 'year',  
3           value_name = 'literacy_rate')  
4       .dropna(axis = 0)  
5       .sort_values(['country', 'year']))
```

	country	year	literacy_rate
1040	Afghanistan	1979	4.987460
9360	Afghanistan	2011	13.000000
6761	Albania	2001	98.252274
8581	Albania	2008	94.681814
9361	Albania	2011	95.691480
...	...	...	...
7227	Zambia	2002	61.839278
8527	Zambia	2007	51.786967
2028	Zimbabwe	1982	71.853928
4628	Zimbabwe	1992	78.517018
9568	Zimbabwe	2011	80.065659

In pandas: melt

Question: How does melt affect the index of the data?

```
1 (litF.melt(id_vars = 'country',
2           var_name = 'year',
3           value_name = 'literacy_rate')
4     .dropna(axis = 0)
5     .sort_values(['country', 'year']))
```

	country	year	literacy_rate
1040	Afghanistan	1979	4.987460
9360	Afghanistan	2011	13.000000
6761	Albania	2001	98.252274
8581	Albania	2008	94.681814
9361	Albania	2011	95.691480
...	...	...	...
7227	Zambia	2002	61.839278
8527	Zambia	2007	51.786967
2028	Zimbabwe	1982	71.853928
4628	Zimbabwe	1992	78.517018
9568	Zimbabwe	2011	80.065659

melt, by default,
makes index
0, ..., n-1

(dropna is removing
rows after we melt)

melt and indexes

We can choose for melt to keep the original index.

Question: what are the implications of having duplicate values in the index?

```
1 (litF.melt(id_vars = 'country',  
2           var_name = 'year',  
3           value_name = 'literacy_rate',  
4           ignore_index = False) ← keep original index  
5           .dropna(axis = 0)  
6           .sort_values(['country', 'year'])))
```

	country	year	literacy_rate	
0	Afghanistan	1979	4.987460	.loc[0]
0	Afghanistan	2011	13.000000	
1	Albania	2001	98.252274	
1	Albania	2008	94.681814	
1	Albania	2011	95.691480	
..	...	...	...	
207	Zambia	2002	61.839278	

207	Zambia	2007	51.786967
208	Zimbabwe	1982	71.853928
208	Zimbabwe	1992	78.517018
208	Zimbabwe	2011	80.065659

melt and indexes

```
1 (litF.melt(id_vars = 'country',
2           var_name = 'year',
3           value_name = 'literacy_rate',
4           ignore_index = False)
5     .dropna(axis = 0)
6     .sort_values(['country', 'year'])).loc[0]
```

	country	year	literacy_rate
0	Afghanistan	1979	4.98746
0	Afghanistan	2011	13.00000

Another example

Consider the following example data:

	id	bp_1	bp_2	hr_1	hr_2
1	1	100	120	60	77
2	2	120	115	75	81
3	3	125	130	80	93

What if we want the data to look like this:

```
# A tibble: 6 × 4
```

	id	stage	bp	hr
	<dbl>	<chr>	<dbl>	<dbl>
1	1	1	100	60
2	1	2	120	77
3	2	1	120	75
4	2	2	115	81
5	3	1	125	80
6	3	2	130	93

In R

```
1 df_3
```

	id	bp_1	bp_2	hr_1	hr_2
1	1	100	120	60	77
2	2	120	115	75	81
3	3	125	130	80	93

```
1 df_3 |>  
2   pivot_longer(cols = -id,  
3                 names_to = c(".value", "stage"),  
4                 names_sep = "_")
```

```
# A tibble: 6 × 4
```

	id	stage	bp	hr
	<dbl>	<chr>	<dbl>	<dbl>
1	1	1	100	60
2	1	2	120	77
3	2	1	120	75
4	2	2	115	81
5	3	1	125	80
6	3	2	130	93

In pandas: wide_to_long

prefixes to keep
as column names

```
1 pd.wide_to_long(df_3, stubnames = ['bp', 'hr'],  
2 i = 'id', j = 'stage', sep = "_")
```

id	stage	bp	hr
1.0	1	100.0	60.0
2.0	1	120.0	75.0
3.0	1	125.0	80.0
1.0	2	120.0	77.0
2.0	2	115.0	81.0
3.0	2	130.0	93.0

↑
id variable
(not pivoting)

what to
name
new
column

(bp)(1)
stub name suffix
 (end up in
 a new
 column)

In pandas: wide_to_long

How does pandas treat the index when using wide_to_long?

```
1 pd.wide_to_long(df_3, stubnames = ['bp', 'hr'],  
2                   i = 'id', j = 'stage', sep = "_")
```

		bp	hr
id	stage		
1.0	1	100.0	60.0
2.0	1	120.0	75.0
3.0	1	125.0	80.0
1.0	2	120.0	77.0
2.0	2	115.0	81.0
3.0	2	130.0	93.0

Pivoting wider

```
1 air_quality
```

```
# A tibble: 1,825 × 3
```

	date.utc		location	value
	<dtm>		<chr>	<dbl>
1	2019-06-18	06:00:00	BETR801	18
2	2019-06-17	08:00:00	BETR801	6.5
3	2019-06-17	07:00:00	BETR801	18.5
4	2019-06-17	06:00:00	BETR801	16
5	2019-06-17	05:00:00	BETR801	7.5
6	2019-06-17	04:00:00	BETR801	7.5
7	2019-06-17	03:00:00	BETR801	7
8	2019-06-17	02:00:00	BETR801	7
9	2019-06-17	01:00:00	BETR801	8
10	2019-06-16	01:00:00	BETR801	15

Pivoting wider

```
1 air_quality |>
2   pivot_wider(id_cols = date.utc,
3               names_from = location,
4               values_from = value) |>
5   arrange(date.utc)
```

A tibble: 1,670 × 3

	date.utc	BETR801	`London Westminster`
	<dtm>	<dbl>	<dbl>
1	2019-04-09 01:00:00	76	NA
2	2019-04-09 02:00:00	91.5	42
3	2019-04-09 03:00:00	99	42
4	2019-04-09 04:00:00	83.5	43
5	2019-04-09 05:00:00	68	44
6	2019-04-09 06:00:00	52.5	44
7	2019-04-09 07:00:00	46.5	43
8	2019-04-09 08:00:00	44	43
9	2019-04-09 09:00:00	43	43
10	2019-04-09 10:00:00	44.5	42

In pandas: pivot

```
1 air_quality.pivot(columns = 'location',  
2                       values = 'value',  
3                       index = 'date.utc')
```

columns to pivot
column of values to use to fill in new columns
columns not to pivot

location BETR801 London Westminster (kept as index for output data)

date.utc		
2019-04-09 01:00:00	76.0	NaN
2019-04-09 02:00:00	91.5	42.0
2019-04-09 03:00:00	99.0	42.0
2019-04-09 04:00:00	83.5	43.0
2019-04-09 05:00:00	68.0	44.0
...	...	...
2019-06-20 20:00:00	NaN	8.0
2019-06-20 21:00:00	NaN	8.0
2019-06-20 22:00:00	NaN	7.0
2019-06-20 23:00:00	NaN	7.0
2019-06-21 00:00:00	NaN	7.0

Class activity

- Work with a neighbor on the class activity
- No need to submit this activity